

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

“Информационная безопасность баз данных”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Реализация БД в рамках СУБД

Выполнила:

Жук Анастасия Валерьевна,
студент группы N3348



(подпись)

Проверил:

Ярцева Наталия Андреевна,
ассистент ФБИТ

(подпись)

Санкт-Петербург
2025г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
Выбор СУБД.....	4
Создание разработанной БД.....	5
1. Создание базы данных.....	5
2. Создание таблиц согласно схеме отношений.....	5
3. Установка внешних ключей. Индексирование таблиц.....	11
4. Заполнение таблиц данными.....	16
Установка взаимосвязей таблиц.....	17
Создание представлений.....	18
ЗАКЛЮЧЕНИЕ.....	19

ВВЕДЕНИЕ

Цель работы: получение навыков по работе с современными системами управления базами данных.

Для достижения поставленной цели необходимо выполнить следующие **задачи**:

- выбрать систему управления базами данных (СУБД);
- создать БД в выбранной СУБД на основе итоговой разработанной схемы отношений из лабораторной работы № 1 и заполнить созданную БД информацией;
- проиндексировать таблицы;
- установить взаимосвязи между таблицами;
- создать представления, составленные в пункте 5 лабораторной 1.

ХОД РАБОТЫ

Выбор СУБД

Для реализации разработанной базы данных выбрана СУБД PostgreSQL.

Обоснование выбора:

- поддержка расширенных функций безопасности (роли, шифрование, аудит), что соответствует тематике дисциплины;
- наличие встроенных средств для создания триггеров, представлений, индексов;
- широкая распространённость в индустрии и поддержка со стороны сообщества;
- совместимость с графическими инструментами администрирования (pgAdmin, DBeaver);
- возможность использования в последующих лабораторных работах (логирование, шифрование, разграничение доступа).

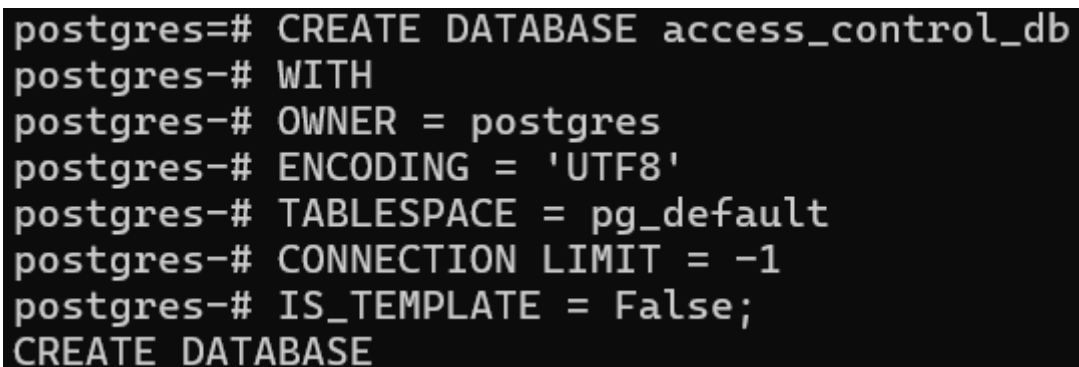
Дистрибутив PostgreSQL можно скачать с официального сайта <https://www.postgresql.org/download/> вместе с графическим клиентом pgAdmin.

Создание разработанной БД

1. Создание базы данных

Код для создания БД выглядит следующим образом:

```
CREATE DATABASE access_control_db  
WITH  
OWNER = postgres  
ENCODING = 'UTF8'  
TABLESPACE = pg_default  
CONNECTION LIMIT = -1  
IS_TEMPLATE = False;
```



```
postgres=# CREATE DATABASE access_control_db  
postgres=# WITH  
postgres=# OWNER = postgres  
postgres=# ENCODING = 'UTF8'  
postgres=# TABLESPACE = pg_default  
postgres=# CONNECTION LIMIT = -1  
postgres=# IS_TEMPLATE = False;  
CREATE DATABASE
```

Рисунок 1 – Успешное создание БД в psql

2. Создание таблиц согласно схеме отношений

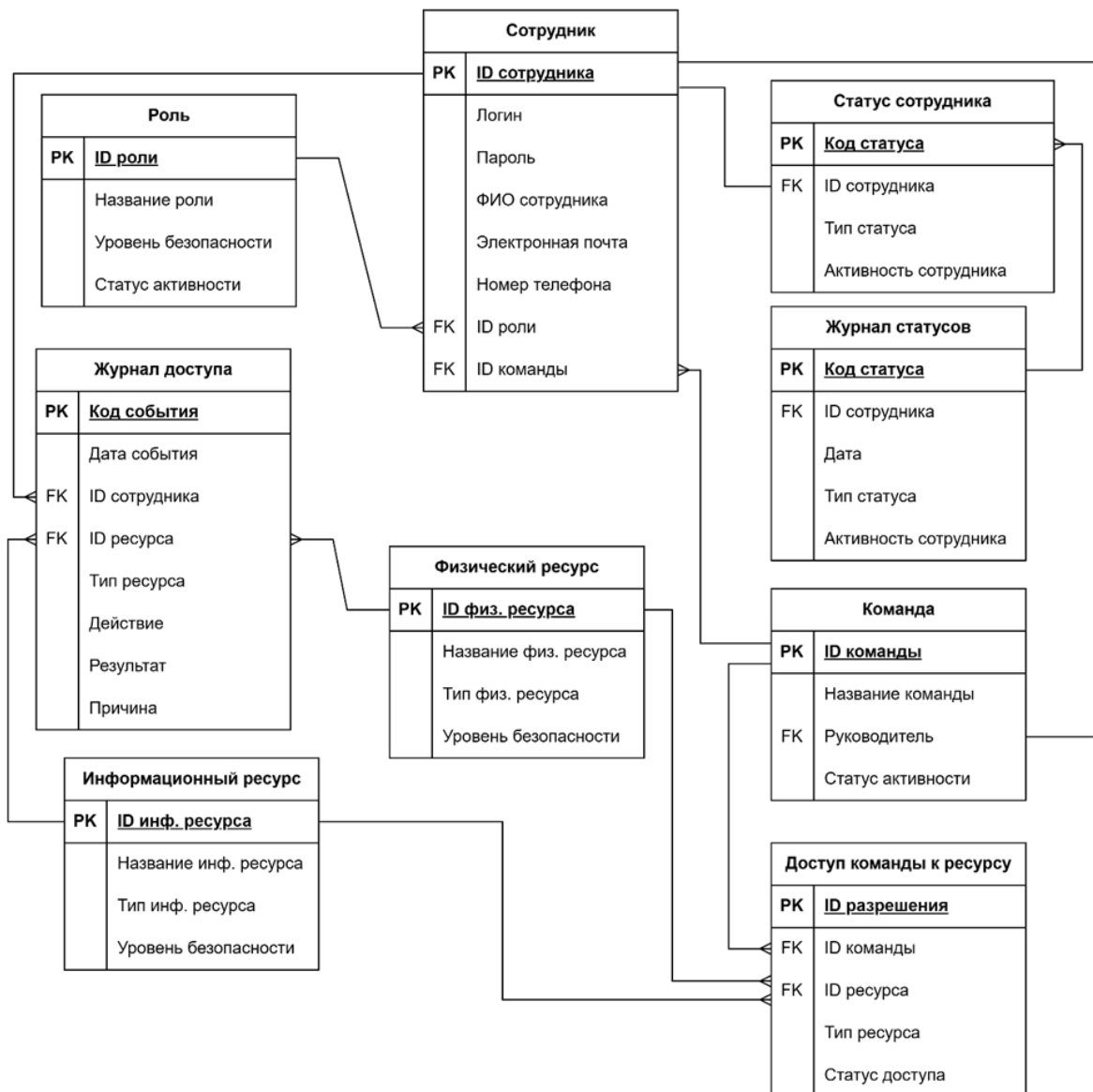


Рисунок 2 – Схема предварительных отношений

Таблица «Роли»

```
CREATE TABLE IF NOT EXISTS public.roles
(
    id_role integer NOT NULL,
    role_name text NOT NULL,
    security_level integer NOT NULL,
    role_active text DEFAULT 'активна' NOT NULL,
    CONSTRAINT role_pkey PRIMARY KEY (id_role),
```

```

        CONSTRAINT chk_security_level CHECK (security_level >= 1 AND
security_level <= 4),
        CONSTRAINT chk_role_active CHECK (role_active IN ('активна',
'неактивна')),
        CONSTRAINT uq_role_name UNIQUE (role_name)
);

```

```

access_control_db=# CREATE TABLE IF NOT EXISTS public.roles
access_control_db=# (
access_control_db(#      id_role integer NOT NULL,
access_control_db(#      role_name text NOT NULL,
access_control_db(#      security_level integer NOT NULL,
access_control_db(#      role_active text DEFAULT 'активна' NOT NULL,
access_control_db(#      CONSTRAINT role_pkey PRIMARY KEY (id_role),
access_control_db(#      CONSTRAINT chk_security_level CHECK (security_level >= 1 AND security_level <= 4),
access_control_db(#      CONSTRAINT chk_role_active CHECK (role_active IN ('активна', 'неактивна')),
access_control_db(#      CONSTRAINT uq_role_name UNIQUE (role_name)
access_control_db(# );
CREATE TABLE

```

Рисунок 3 – Успешное создание таблицы «Роли»

Таблица «Команда»

```

CREATE TABLE IF NOT EXISTS public.teams
(
    id_team integer NOT NULL,
    team_name text NOT NULL,
    leader_id integer,
    team_active text DEFAULT 'активна' NOT NULL,
    CONSTRAINT team_pkey PRIMARY KEY (id_team),
    CONSTRAINT chk_team_active CHECK (team_active IN ('активна',
'неактивна')),
    CONSTRAINT uq_team_name UNIQUE (team_name)
);

```

```

access_control_db=# CREATE TABLE IF NOT EXISTS public.teams
access_control_db=# (
access_control_db(#      id_team integer NOT NULL,
access_control_db(#      team_name text NOT NULL,
access_control_db(#      leader_id integer NOT NULL,
access_control_db(#      team_active text DEFAULT 'активна' NOT NULL,
access_control_db(#      CONSTRAINT team_pkey PRIMARY KEY (id_team),
access_control_db(#      CONSTRAINT chk_team_active CHECK (team_active IN ('активна', 'неактивна')),
access_control_db(#      CONSTRAINT uq_team_name UNIQUE (team_name)
access_control_db(# );
CREATE TABLE

```

Рисунок 4 – Успешное создание таблицы «Команды»

Таблица «Сотрудники»

```

CREATE TABLE IF NOT EXISTS public.employees

```

```
(
    id_employee integer NOT NULL,
    login text NOT NULL,
    password text NOT NULL,
    full_name text NOT NULL,
    email text,
    phone text,
    id_role integer NOT NULL,
    id_team integer,
    CONSTRAINT employee_pkey PRIMARY KEY (id_employee),
    CONSTRAINT uq_login UNIQUE (login)
);
```

```
access_control_db=# CREATE TABLE IF NOT EXISTS public.employees
access_control_db=# (
access_control_db(#      id_employee integer NOT NULL,
access_control_db(#      login text NOT NULL,
access_control_db(#      password text NOT NULL,
access_control_db(#      full_name text NOT NULL,
access_control_db(#      email text,
access_control_db(#      phone text,
access_control_db(#      id_role integer NOT NULL,
access_control_db(#      id_team integer,
access_control_db(#      CONSTRAINT employee_pkey PRIMARY KEY (id_employee),
access_control_db(#      CONSTRAINT uq_login UNIQUE (login)
access_control_db(# );
CREATE TABLE
```

Рисунок 5 – Успешное создание таблицы «Сотрудники»

Таблица «Статусы сотрудников»

```
CREATE TABLE IF NOT EXISTS public.employees_statuses
```

```
(
    status_code integer NOT NULL,
    id_employee integer NOT NULL,
    status_type text NOT NULL,
    employee_active integer NOT NULL,
    CONSTRAINT employee_status_pkey PRIMARY KEY (status_code),
    CONSTRAINT chk_status_type CHECK (status_type IN ('Активная
    работа', 'Отпуск', 'Больничный', 'Командировка', 'Декретный
    отпуск', 'Уволен')),
    CONSTRAINT chk_employee_active CHECK (employee_active IN (0,
    1))
```


);

```
access_control_db=# CREATE TABLE IF NOT EXISTS public.employees_statuses
access_control_db=# (
access_control_db(#      status_code integer NOT NULL,
access_control_db(#      id_employee integer NOT NULL,
access_control_db(#      status_type text NOT NULL,
access_control_db(#      employee_active integer NOT NULL,
access_control_db(#      CONSTRAINT employee_status_pkey PRIMARY KEY (status_code),
access_control_db(#      CONSTRAINT chk_status_type CHECK (status_type IN ('Активная работа', 'Отпуск', 'Больничный',
access_control_db(#      'Командировка', 'Декретный отпуск', 'Уволен')),
access_control_db(#      CONSTRAINT chk_employee_active CHECK (employee_active IN (0, 1))
access_control_db=# );
CREATE TABLE
```

Рисунок 6 – Успешное создание таблицы «Статусы сотрудников»

Таблица «Журнал статусов»

```
CREATE TABLE IF NOT EXISTS public.status_log
(
    log_id integer NOT NULL,
    status_code integer NOT NULL,
    record_date DATE NOT NULL,
    status_type text NOT NULL,
    employee_active integer NOT NULL,
    CONSTRAINT status_log_pkey PRIMARY KEY (log_id),
    CONSTRAINT chk_employee_active_log CHECK (employee_active IN
(0, 1))
);
```

```
access_control_db=# CREATE TABLE IF NOT EXISTS public.status_log
access_control_db=# (
access_control_db(#      log_id integer NOT NULL,
access_control_db(#      status_code integer NOT NULL,
access_control_db(#      record_date DATE NOT NULL,
access_control_db(#      status_type text NOT NULL,
access_control_db(#      employee_active integer NOT NULL,
access_control_db(#      CONSTRAINT status_log_pkey PRIMARY KEY (log_id),
access_control_db(#      CONSTRAINT chk_employee_active_log CHECK (employee_active IN (0, 1))
access_control_db=# );
CREATE TABLE
```

Рисунок 7 – Успешное создание таблицы «Журнал статусов»

Таблица «Физические ресурсы»

```
CREATE TABLE IF NOT EXISTS public.physical_resources
(
    id_physical_resource integer NOT NULL,
    resource_name text NOT NULL,
    security_level integer NOT NULL,
```

```

        CONSTRAINT physical_resource_pkey PRIMARY KEY
(id_physical_resource),
    CONSTRAINT chk_phys_sec_level CHECK (security_level >= 1 AND
security_level <= 4)
);

```

```

access_control_db=# CREATE TABLE IF NOT EXISTS public.physical_resources
access_control_db=# (
access_control_db(#      id_physical_resource integer NOT NULL,
access_control_db(#      resource_name text NOT NULL,
access_control_db(#      security_level integer NOT NULL,
access_control_db(#      CONSTRAINT physical_resource_pkey PRIMARY KEY (id_physical_resource),
access_control_db(#      CONSTRAINT chk_phys_sec_level CHECK (security_level >= 1 AND security_level <= 4)
access_control_db(# );
CREATE TABLE

```

Рисунок 8 – Успешное создание таблицы «Физические ресурсы»

Таблица «Информационные ресурсы»

```

CREATE TABLE IF NOT EXISTS public.information_resources
(
    id_information_resource integer NOT NULL,
    resource_name text NOT NULL,
    security_level integer NOT NULL,
    CONSTRAINT information_resource_pkey PRIMARY KEY
(id_information_resource),
    CONSTRAINT chk_info_sec_level CHECK (security_level >= 1 AND
security_level <= 4)
);

```

```

access_control_db=# CREATE TABLE IF NOT EXISTS public.information_resources
access_control_db=# (
access_control_db(#      id_information_resource integer NOT NULL,
access_control_db(#      resource_name text NOT NULL,
access_control_db(#      security_level integer NOT NULL,
access_control_db(#      CONSTRAINT information_resource_pkey PRIMARY KEY (id_information_resource),
access_control_db(#      CONSTRAINT chk_info_sec_level CHECK (security_level >= 1 AND security_level <= 4)
access_control_db(# );
CREATE TABLE

```

Рисунок 9 – Успешное создание таблицы «Информационные ресурсы»

Таблица «Журнал доступа»

```

CREATE TABLE IF NOT EXISTS public.access_log
(
    event_id integer NOT NULL,
    id_employee integer NOT NULL,
    id_physical_resource integer,

```

```

    id_information_resource integer,
    resource_type text NOT NULL,
    event_date DATE NOT NULL,
    action text NOT NULL,
    result text NOT NULL,
    reason text,
    CONSTRAINT access_log_pkey PRIMARY KEY (event_id),
    CONSTRAINT chk_resource_type CHECK (resource_type IN ('phys',
'inf')),
    CONSTRAINT chk_result CHECK (result IN ('успешно', 'отказано')),
    CONSTRAINT chk_exclusive_resource CHECK (
        (id_physical_resource IS NOT NULL AND id_information_resource
IS NULL) OR
        (id_physical_resource IS NULL AND id_information_resource IS
NOT NULL)
    )
);

```

```

access_control_db=# CREATE TABLE IF NOT EXISTS public.access_log
access_control_db=# (
access_control_db(#     event_id integer NOT NULL,
access_control_db(#     id_employee integer NOT NULL,
access_control_db(#     id_physical_resource integer,
access_control_db(#     id_information_resource integer,
access_control_db(#     resource_type text NOT NULL,
access_control_db(#     event_date DATE NOT NULL,
access_control_db(#     action text NOT NULL,
access_control_db(#     result text NOT NULL,
access_control_db(#     reason text,
access_control_db(#     CONSTRAINT access_log_pkey PRIMARY KEY (event_id),
access_control_db(#     CONSTRAINT chk_resource_type CHECK (resource_type IN ('phys', 'inf')),
access_control_db(#     CONSTRAINT chk_result CHECK (result IN ('успешно', 'отказано')),
access_control_db(#     CONSTRAINT chk_exclusive_resource CHECK (
access_control_db(#         (id_physical_resource IS NOT NULL AND id_information_resource IS NULL) OR
access_control_db(#         (id_physical_resource IS NULL AND id_information_resource IS NOT NULL)
access_control_db(#     )
access_control_db=# );
CREATE TABLE

```

Рисунок 10 – Успешное создание таблицы «Журнал доступа»

Таблица «Доступ команд к ресурсам»

```

CREATE TABLE IF NOT EXISTS public.teams_resources_access
(
    permission_id integer NOT NULL,
    id_team integer NOT NULL,
    id_physical_resource integer,
    id_information_resource integer,
    resource_type text NOT NULL,

```

```

    access_status text DEFAULT 'чтение' NOT NULL,
    permission_active text DEFAULT 'активен' NOT NULL,
    CONSTRAINT team_resource_access_pkey PRIMARY KEY
(permission_id),
    CONSTRAINT chk_access_status CHECK (access_status IN ('чтение',
'запись', 'админ')),
    CONSTRAINT chk_permission_active CHECK (permission_active IN
('активна', 'неактивна')),
    CONSTRAINT chk_res_type CHECK (resource_type IN ('phys', 'inf')),
    CONSTRAINT chk_excl_res CHECK (
(id_physical_resource IS NOT NULL AND id_information_resource
IS NULL) OR
(id_physical_resource IS NULL AND id_information_resource IS
NOT NULL)
)
);

```

```

access_control_db=# CREATE TABLE IF NOT EXISTS public.teams_resources_access
access_control_db=# (
access_control_db=#     permission_id integer NOT NULL,
access_control_db=#     id_team integer NOT NULL,
access_control_db=#     id_physical_resource integer,
access_control_db=#     id_information_resource integer,
access_control_db=#     resource_type text NOT NULL,
access_control_db=#     access_status text DEFAULT 'чтение' NOT NULL,
access_control_db=#     permission_active text DEFAULT 'активен' NOT NULL,
access_control_db=#     CONSTRAINT team_resource_access_pkey PRIMARY KEY (permission_id),
access_control_db=#     CONSTRAINT chk_access_status CHECK (access_status IN ('чтение', 'запись', 'админ')),
access_control_db=#     CONSTRAINT chk_permission_active CHECK (permission_active IN ('активна', 'неактивна')),
access_control_db=#     CONSTRAINT chk_res_type CHECK (resource_type IN ('phys', 'inf')),
access_control_db=#     CONSTRAINT chk_excl_res CHECK (
access_control_db=#         (id_physical_resource IS NOT NULL AND id_information_resource IS NULL) OR
access_control_db=#         (id_physical_resource IS NULL AND id_information_resource IS NOT NULL)
access_control_db=#     )
access_control_db=# );
access_control_db=# CREATE TABLE

```

Рисунок 11 – Успешное создание таблицы «Доступ команд к ресурсам»

3. Установка взаимосвязей таблиц. Индексирование таблиц

Добавим дополнительные ограничения на отношения, соответствующие внешним ключам и проиндексируем их, чтобы запросы при объединении таблиц выполнялись быстрее.

Сотрудник – Роль

```

ALTER TABLE IF EXISTS public.employees
ADD CONSTRAINT fk_employee_role FOREIGN KEY (id_role)
REFERENCES public.roles (id_role) MATCH SIMPLE
ON UPDATE CASCADE

```

*ON DELETE RESTRICT
NOT VALID;
CREATE INDEX IF NOT EXISTS fki_employee_role
ON public.employees(id_role);*

```
access_control_db=# ALTER TABLE IF EXISTS public.employees
access_control_db=# ADD CONSTRAINT fk_employee_role FOREIGN KEY (id_role)
access_control_db=# REFERENCES public.roles (id_role) MATCH SIMPLE
access_control_db=# ON UPDATE CASCADE
access_control_db=# ON DELETE RESTRICT
access_control_db=# NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_employee_role
access_control_db=# ON public.employees(id_role);
CREATE INDEX
```

Рисунок 12 – Внешний ключ + индекс. Сотрудник – Роль

Сотрудник – Команда

*ALTER TABLE IF EXISTS public.employees
ADD CONSTRAINT fk_employee_team FOREIGN KEY (id_team)
REFERENCES public.teams (id_team) MATCH SIMPLE
ON UPDATE SET NULL
ON DELETE SET NULL
NOT VALID;
CREATE INDEX IF NOT EXISTS fki_employee_team
ON public.employees(id_team);*

```
access_control_db=# ALTER TABLE IF EXISTS public.employees
access_control_db=# ADD CONSTRAINT fk_employee_team FOREIGN KEY (id_team)
access_control_db=# REFERENCES public.teams (id_team) MATCH SIMPLE
access_control_db=# ON UPDATE SET NULL
access_control_db=# ON DELETE SET NULL
access_control_db=# NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_employee_team
access_control_db=# ON public.employees(id_team);
CREATE INDEX
```

Рисунок 13 – Внешний ключ + индекс. Сотрудник – Команда

Команда – Сотрудник (Руководитель)

*ALTER TABLE IF EXISTS public.teams
ADD CONSTRAINT fk_team_leader FOREIGN KEY (leader_id)
REFERENCES public.employees (id_employee) MATCH SIMPLE
ON UPDATE SET NULL*

```

ON DELETE SET NULL
NOT VALID;
CREATE INDEX IF NOT EXISTS fki_team_leader
ON public.teams(leader_id);

```

```

access_control_db=# ALTER TABLE IF EXISTS public.teams
access_control_db=# ADD CONSTRAINT fk_team_leader FOREIGN KEY (leader_id)
access_control_db=# REFERENCES public.employees (id_employee) MATCH SIMPLE
access_control_db=# ON UPDATE SET NULL
access_control_db=# ON DELETE SET NULL
access_control_db=# NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_leader
access_control_db=# ON public.teams(leader_id);
CREATE INDEX

```

Рисунок 14 – Внешний ключ + индекс. Команда – Сотрудник
(Руководитель)

Статус сотрудника – Сотрудник

```

ALTER TABLE IF EXISTS public.employees_statuses
ADD CONSTRAINT fk_status_employee FOREIGN KEY
(id_employee)
REFERENCES public.employees (id_employee) MATCH SIMPLE
ON UPDATE CASCADE
ON DELETE CASCADE
NOT VALID;
CREATE INDEX IF NOT EXISTS fki_status_employee
ON public.employees_statuses(id_employee);

```

```

access_control_db=# ALTER TABLE IF EXISTS public.employees_statuses
access_control_db=# ADD CONSTRAINT fk_status_employee FOREIGN KEY (id_employee)
access_control_db=# REFERENCES public.employees (id_employee) MATCH SIMPLE
access_control_db=# ON UPDATE CASCADE
access_control_db=# ON DELETE CASCADE
access_control_db=# NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_status_employee
access_control_db=# ON public.employees_statuses(id_employee);
CREATE INDEX

```

Рисунок 15 – Внешний ключ + индекс. Статус сотрудника – Сотрудник

Журнал статусов – Статус сотрудника

```

ALTER TABLE IF EXISTS public.status_log
ADD CONSTRAINT fk_log_status FOREIGN KEY (status_code)

```

```

REFERENCES public.employees_statuses (status_code) MATCH
SIMPLE
ON UPDATE CASCADE
ON DELETE CASCADE
NOT VALID;
CREATE INDEX IF NOT EXISTS fki_log_status
ON public.status_log(status_code);

```

```

access_control_db=# ALTER TABLE IF EXISTS public.status_log
access_control_db=# ADD CONSTRAINT fk_log_status FOREIGN KEY (status_code)
access_control_db=# REFERENCES public.employees_statuses (status_code) MATCH SIMPLE
access_control_db=# ON UPDATE CASCADE
access_control_db=# ON DELETE CASCADE
access_control_db=# NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_log_status
access_control_db=# ON public.status_log(status_code);
CREATE INDEX

```

Рисунок 16 – Внешний ключ + индекс. Журнал статусов – Статус
сотрудника

Журнал доступа – Сотрудник

```

ALTER TABLE IF EXISTS public.access_log
ADD CONSTRAINT fk_access_employee FOREIGN KEY
(id_employee)
REFERENCES public.employees (id_employee) MATCH SIMPLE
ON UPDATE CASCADE
ON DELETE CASCADE
NOT VALID;
CREATE INDEX IF NOT EXISTS fki_access_employee
ON public.access_log(id_employee);

```

```

access_control_db=# ALTER TABLE IF EXISTS public.access_log
access_control_db=# ADD CONSTRAINT fk_access_employee FOREIGN KEY (id_employee)
access_control_db=# REFERENCES public.employees (id_employee) MATCH SIMPLE
access_control_db=# ON UPDATE CASCADE
access_control_db=# ON DELETE CASCADE
access_control_db=# NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_access_employee
access_control_db=# ON public.access_log(id_employee);
CREATE INDEX

```

Рисунок 17 – Внешний ключ + индекс. Журнал доступа – Сотрудник

Доступ команды к ресурсам – Команда

```

ALTER TABLE IF EXISTS public.teams_resources_access
  ADD CONSTRAINT fk_team_res_team FOREIGN KEY (id_team)
  REFERENCES public.teams (id_team) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE
  NOT VALID;
CREATE INDEX IF NOT EXISTS fki_team_res_team
  ON public.teams_resources_access(id_team);

```

```

access_control_db=# ALTER TABLE IF EXISTS public.teams_resources_access
access_control_db=#     ADD CONSTRAINT fk_team_res_team FOREIGN KEY (id_team)
access_control_db=#     REFERENCES public.teams (id_team) MATCH SIMPLE
access_control_db=#     ON UPDATE CASCADE
access_control_db=#     ON DELETE CASCADE
access_control_db=#     NOT VALID;
ALTER TABLE
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_res_team
access_control_db=#     ON public.teams_resources_access(id_team);
CREATE INDEX

```

Рисунок 18 – Внешний ключ + индекс. Доступ команды к ресурсам – Команда

Журнал доступа – Физический ресурс (условный FK – только если ресурс типа 'phys')

Нужно отметить, что из-за разделения на два типа ресурсов, полный FK невозможен. Вместо этого создаем индексы для ускорения JOIN.

```

CREATE INDEX IF NOT EXISTS fki_access_phys
  ON public.access_log(id_physical_resource);

```

```

access_control_db=# CREATE INDEX IF NOT EXISTS fki_access_phys
access_control_db=#     ON public.access_log(id_physical_resource);
CREATE INDEX

```

Рисунок 19 – Внешний ключ + индекс. Журнал доступа – Физический ресурс

Журнал доступа – Информационный ресурс (условный FK — только если ресурс типа 'inf')

```

CREATE INDEX IF NOT EXISTS fki_access_info
  ON public.access_log(id_information_resource);

```



```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_access_info
access_control_db=# ON public.access_log(id_information_resource);
CREATE INDEX
```

Рисунок 20 – Внешний ключ + индекс. Журнал доступа – Информационный ресурс

Доступ команды к ресурсам – Физический ресурс

```
CREATE INDEX IF NOT EXISTS fki_team_res_phys
ON public.teams_resources_access(id_physical_resource);
```

```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_res_phys
access_control_db=# ON public.teams_resources_access(id_physical_resource);
CREATE INDEX
```

Рисунок 21 – Внешний ключ + индекс. Доступ команды к ресурсам – Физический ресурс

Доступ команды к ресурсам – Информационный ресурс

```
CREATE INDEX IF NOT EXISTS fki_team_res_info
ON public.teams_resources_access(id_information_resource);
```

```
access_control_db=# CREATE INDEX IF NOT EXISTS fki_team_res_info
access_control_db=# ON public.teams_resources_access(id_information_resource);
CREATE INDEX
```

Рисунок 22 – Внешний ключ + индекс. Доступ команды к ресурсам – Информационный ресурс

4. Заполнение таблиц данными

Важно отметить значения некоторых столбцов:

- security_level от 1 до 4 (где 1 – низкий, 2 – средний, 3 – высокий, 4 – критический),
- resource_type – “phys” или “inf”,
- role_active, team_active – “активна” / “неактивна”,
- employee_active – 0 или 1 (где 0 – сотрудник в определенный день работает, 1 – не работает),
- access_status – “чтение”, “запись”, “админ” (“админ” – чтение + запись),
- permission_active — “активна” / “неактивна”.

Таблица **public.roles**

```
INSERT INTO public.roles (id_role, role_name, security_level,
role_active) VALUES
(1, 'Стажер', 1, 'активна'),
(2, 'Разработчик', 2, 'активна'),
(3, 'Тимлид', 3, 'активна'),
(4, 'DevOps-инженер', 3, 'активна'),
(5, 'Администратор ИБ', 4, 'активна'),
(6, 'Служба безопасности', 4, 'активна'),
(7, 'Аудитор', 3, 'активна'),
(8, 'Руководитель отдела', 3, 'активна');
```

Таблица **public.teams**

```
INSERT INTO public.teams (id_team, team_name, leader_id,
team_active) VALUES
(1, 'Frontend Team', NULL, 'активна'),
(2, 'Backend Team', NULL, 'активна'),
(3, 'QA Team', NULL, 'активна'),
(4, 'DevSecOps', NULL, 'активна'),
(5, 'Инфраструктура', NULL, 'активна'),
(6, 'Аналитика', NULL, 'активна'),
(7, 'Архивная команда', NULL, 'неактивна');
```

Таблица **public.employees**

В этой таблице хранятся пароли, но так как пароли не могут храниться в БД в открытом виде, необходимо их захэшировать. Сделать это можно, например, с помощью функции `crypt()`. Сначала нужно активировать модуль `pgcrypto` (выполняется один раз):

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
INSERT INTO public.employees (id_employee, login, password,
full_name, email, phone, id_role, id_team) VALUES
(1, 'ivanov_i', crypt('password1', gen_salt('bf')), 'Иванов Иван
Иванович', 'ivanov@company.com', '+79001112233', 2, 1),
(2, 'petrov_p', crypt('password2', gen_salt('bf')), 'Петров Петр
Петрович', 'petrov@company.com', '+79001112234', 3, 2),
```

```
(3, 'sidorov_s', crypt('password3', gen_salt('bf')), 'Сидоров Сидор
Сидорович', 'sidorov@company.com', '+79001112235', 4, 4),
(4, 'admin_ib', crypt('password4', gen_salt('bf')), 'Козлов Геннадий
Андреевич', 'admin_ib@company.com', '+79001112236', 5, NULL),
(5, 'sec_officer', crypt('password5', gen_salt('bf')), 'Васьков Владимир
Васильевич', 'sec@company.com', '+79001112237', 6, NULL),
(6, 'auditor1', crypt('password6', gen_salt('bf')), 'Дементьев Сергей
Данилович', 'audit@company.com', '+79001112238', 7, NULL),
(7, 'manager_a', crypt('password7', gen_salt('bf')), 'Кривова Анна
Валерьевна', 'manager@company.com', '+79001112239', 8, 1),
(8, 'vacation_user', crypt('password8', gen_salt('bf')), 'Прокофьев Денис
Иванович', 'ex@company.com', NULL, 2, NULL);
```

Обновление leader_id в public.teams

```
UPDATE public.teams SET leader_id = 7 WHERE id_team = 1;
UPDATE public.teams SET leader_id = 2 WHERE id_team = 2;
UPDATE public.teams SET leader_id = 3 WHERE id_team = 4;
```

Таблица public.employees_statuses

```
INSERT INTO public.employees_statuses (status_code, id_employee,
status_type, employee_active) VALUES
(1, 1, 'Активная работа', 1),
(2, 2, 'Активная работа', 1),
(3, 3, 'Активная работа', 1),
(4, 4, 'Активная работа', 1),
(5, 5, 'Активная работа', 1),
(6, 6, 'Активная работа', 1),
(7, 7, 'Активная работа', 1),
(8, 8, 'Уволен', 0);
```

Таблица public.status_log

```
INSERT INTO public.status_log (log_id, status_code, record_date,
status_type, employee_active) VALUES
(1, 1, '2025-04-01', 'Активная работа', 1),
(2, 8, '2025-03-15', 'Больничный', 1),
(3, 8, '2025-04-01', 'Уволен', 0),
(4, 2, '2025-04-01', 'Активная работа', 1),
```

(5, 3, '2025-04-01', 'Активная работа', 1),
(6, 4, '2025-04-01', 'Активная работа', 1),
(7, 5, '2025-04-01', 'Активная работа', 1),
(8, 6, '2025-04-01', 'Активная работа', 1);

Таблица public.physical_resources

```
INSERT INTO public.physical_resources (id_physical_resource,  
resource_name, security_level) VALUES  
(1, 'Серверная комната А', 4),  
(2, 'Лаборатория тестирования', 2),  
(3, 'Кабинет администратора ИБ', 4),  
(4, 'Общее офисное помещение', 1),  
(5, 'Архив документов', 3),  
(6, 'Комнату для переговоров', 2),  
(7, 'Сейф с ключами', 4);
```

Таблица public.information_resources

```
INSERT INTO public.information_resources (id_information_resource,  
resource_name, security_level) VALUES  
(1, 'База данных клиентов', 4),  
(2, 'CI/CD система', 3),  
(3, 'Внутренний портал', 1),  
(4, 'Репозиторий исходного кода', 3),  
(5, 'Журнал аудита', 4),  
(6, 'HR-система', 2),  
(7, 'Шифрованный архив', 4);
```

Таблица public.access_log

```
INSERT INTO public.access_log (  
event_id, id_employee, id_physical_resource, id_information_resource,  
resource_type, event_date, action, result, reason  
) VALUES  
(1, 1, NULL, 3, 'inf', '2025-04-01', 'вход', 'успешно', NULL),  
(2, 2, NULL, 1, 'inf', '2025-04-01', 'чтение', 'отказано', 'уровень роли  
ниже уровня ресурса'),  
(3, 8, NULL, 3, 'inf', '2025-04-01', 'вход', 'отказано', 'статус  
"Уволен")
```

(4, 5, NULL, 5, 'inf', '2025-04-01', 'чтение', 'успешно', NULL),
 (5, 1, 4, NULL, 'phys', '2025-04-01', 'вход в офис', 'успешно', NULL),
 (6, 2, 1, NULL, 'phys', '2025-04-01', 'доступ к серверной', 'отказано',
 'уровень роли ниже уровня ресурса'),
 (7, 4, 3, NULL, 'phys', '2025-04-01', 'доступ к кабинету', 'успешно',
 NULL),
 (8, 3, NULL, 2, 'inf', '2025-04-01', 'запуск pipeline', 'успешно', NULL);

Таблица public.teams_resources_access

```

INSERT INTO public.teams_resources_access (
    permission_id,    id_team,    id_physical_resource,
    id_information_resource,
    resource_type, access_status, permission_active
) VALUES
(1, 1, NULL, 4, 'inf', 'чтение', 'активна'),
(2, 2, NULL, 4, 'inf', 'чтение', 'активна'),
(3, 2, NULL, 2, 'inf', 'запись', 'активна'),
(4, 4, NULL, 1, 'inf', 'админ', 'активна'),
(5, 4, NULL, 5, 'inf', 'чтение', 'активна'),
(6, 3, NULL, 2, 'inf', 'чтение', 'активна'),
(7, 3, NULL, 3, 'inf', 'чтение', 'активна'),
(8, 1, 4, NULL, 'phys', 'чтение', 'активна');
  
```

Содержимое таблиц

```

SELECT * FROM access_log;
SELECT * FROM employees;
SELECT * FROM employees_statuses;
SELECT * FROM information_resources;
SELECT * FROM physical_resources;
SELECT * FROM roles;
SELECT * FROM status_log;
SELECT * FROM teams;
SELECT * FROM teams_resources_access;
  
```

Создание представлений

Создадим представления, составленные в пункте 5 лабораторной работы №1, для удобного просмотра информации пользователями.

Представление 1. «Общий обзор прав доступа»

Список атрибутов, отображаемых в рамках представления «Общий обзор прав доступа»:

- Логин (отношение «employees»);
- ФИО (отношение «employees»);
- Название роли (отношение «roles»);
- Уровень безопасности роли (отношение «roles»);
- Название команды (отношение «teams»);
- Тип ресурса (отношение «teams_resources_access»);
- Название ресурса (отношение «physical_resources» / «information_resources»);
- Уровень безопасности ресурса (отношение «physical_resources» / «information_resources»).

Исходя из текущих обозначений атрибутов в БД, SQL-код для создания представления может выглядеть следующим образом:

```
CREATE VIEW public.overview_access_rights
AS
SELECT
    e.login,
    e.full_name,
    r.role_name,
    r.security_level AS role_security_level,
    t.team_name,
    COALESCE('Physical', 'Informational') AS resource_type,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    COALESCE(pr.security_level, ir.security_level) AS
resource_security_level
FROM public.employees e
JOIN public.roles r ON e.id_role = r.id_role
LEFT JOIN public.teams t ON e.id_team = t.id_team
```

```

LEFT JOIN public.teams_resources_access tra ON t.id_team =
tra.id_team
LEFT JOIN public.physical_resources pr ON tra.id_physical_resource =
pr.id_physical_resource AND tra.resource_type = 'phys'
LEFT JOIN public.information_resources ir ON
tra.id_information_resource = ir.id_information_resource AND
tra.resource_type = 'inf';

```

Представление 2. «Попытки нарушения политик безопасности»

Список атрибутов, отображаемых в рамках представления «Попытки нарушения политик безопасности»:

- Логин (отношение «employees»);
- Название роли (отношение «roles»);
- Уровень безопасности роли (отношение «roles»);
- Название ресурса (отношение «physical_resources» / «information_resources»);
- Уровень безопасности ресурса (отношение «physical_resources» / «information_resources»);
- Тип статуса (отношение «employees_statuses»);
- Дата записи (отношение «access_log»);
- Результат (отношение «access_log»);
- Причина (отношение «access_log»).

```

CREATE VIEW public.policy_violation_attempts
AS
SELECT
    e.login,
    r.role_name,
    r.security_level AS role_sec_level,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    COALESCE(pr.security_level, ir.security_level) AS resource_sec_level,
    es.status_type,
    al.event_date,
    al.result,
    al.reason
FROM public.access_log al
JOIN public.employees e ON al.id_employee = e.id_employee

```

```

JOIN public.roles r ON e.id_role = r.id_role
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
LEFT JOIN public.physical_resources pr ON al.id_physical_resource =
pr.id_physical_resource
LEFT JOIN public.information_resources ir ON
al.id_information_resource = ir.id_information_resource
WHERE
(r.security_level < COALESCE(pr.security_level, ir.security_level))
OR es.status_type IN ('Отпуск', 'Больничный', 'Уволен');

```

Представление 3. «Состав и статусы команды»

Список атрибутов, отображаемых в рамках представления «Состав и статусы команды»:

- Название команды (отношение «teams»);
- ФИО сотрудника (отношение «employees»);
- Логин (отношение «employees»);
- Название роли (отношение «roles»);
- Тип статуса (отношение «employees_statuses»);
- Активность (отношение «employees_statuses»);
- Признак прогула (вычисляемый атрибут).

```

CREATE VIEW public.team_members_and_status
AS
SELECT
    t.team_name,
    e.full_name,
    e.login,
    r.role_name,
    es.status_type,
    es.employee_active,
    CASE
        WHEN es.status_type = 'Активная работа' AND
es.employee_active = 0 THEN TRUE
        ELSE FALSE
    END AS is_absentee
FROM public.teams t
JOIN public.employees e ON t.id_team = e.id_team

```



```
JOIN public.roles r ON e.id_role = r.id_role
JOIN public.employees_statuses es ON e.id_employee = es.id_employee;
```

Представление 4. «Ресурсы, доступные команде»

Список атрибутов, отображаемых в рамках представления «Ресурсы, доступные команде»:

- Название команды (отношение «teams»);
- Тип ресурса (отношение «teams_resources_access»);
- Название ресурса (отношение «physical_resources» / «information_resources»);
- Уровень безопасности ресурса (отношение «physical_resources» / «information_resources»);
- Статус доступа (отношение «teams_resources_access»).

```
CREATE VIEW public.team_available_resources
AS
SELECT
    t.team_name,
    tra.resource_type,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    COALESCE(pr.security_level, ir.security_level) AS security_level,
    tra.access_status
FROM public.teams t
JOIN public.teams_resources_access tra ON t.id_team = tra.id_team
LEFT JOIN public.physical_resources pr ON tra.id_physical_resource =
pr.id_physical_resource
LEFT JOIN public.information_resources ir ON
tra.id_information_resource = ir.id_information_resource;
```

Представление 5. «Мои текущие права доступа»

Список атрибутов, отображаемых в рамках представления «Мои текущие права доступа»:

- Логин (отношение «employees»);
- ФИО (отношение «employees»);
- Название роли (отношение «roles»);
- Уровень безопасности роли (отношение «roles»);
- Название команды (отношение «teams»);

- Тип ресурса (отношение «teams_resources_access»);
- Название ресурса (отношение «physical_resources» / «information_resources»).

```
CREATE VIEW public.my_current_access
AS
SELECT
    e.login,
    e.full_name,
    r.role_name,
    r.security_level AS role_sec_level,
    t.team_name,
    COALESCE('Physical', 'Informational') AS resource_type,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name
FROM public.employees e
JOIN public.roles r ON e.id_role = r.id_role
LEFT JOIN public.teams t ON e.id_team = t.id_team
LEFT JOIN public.teams_resources_access tra ON t.id_team =
tra.id_team
LEFT JOIN public.physical_resources pr ON tra.id_physical_resource =
pr.id_physical_resource AND tra.resource_type = 'phys'
LEFT JOIN public.information_resources ir ON
tra.id_information_resource = ir.id_information_resource AND
tra.resource_type = 'inf';
```

Представление 6. «Мой статус и активность»

Список атрибутов, отображаемых в рамках представления «Мой статус и активность»:

- Логин (отношение «employees»);
- Текущий тип статуса (отношение «employees_statuses»);
- Активность (отношение «employees_statuses»);
- История: Дата — Тип статуса (отношение «status_log»).

```
CREATE VIEW public.my_status_and_activity
AS
SELECT
    e.login,
    es.status_type AS current_status,
```

```

    es.employee_active,
    sl.record_date,
    sl.status_type AS historical_status
FROM public.employees e
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
JOIN public.status_log sl ON es.status_code = sl.status_code;

```

Представление 7. «Подозрительные попытки доступа»

Список атрибутов, отображаемых в рамках представления «Подозрительные попытки доступа»:

- Логин (отношение «employees»)
- Тип статуса (отношение «employees_statuses»)
- Название ресурса (отношение «physical_resources» / «information_resources»)
- Тип ресурса (отношение «access_log»)
- Дата записи (отношение «access_log»)
- Действие (отношение «access_log»)
- Результат (отношение «access_log»)
- Причина (отношение «access_log»)

```

CREATE VIEW public.suspicious_access_attempts
AS
SELECT
    e.login,
    es.status_type,
    COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
    al.resource_type,
    al.event_date,
    al.action,
    al.result,
    al.reason
FROM public.access_log al
JOIN public.employees e ON al.id_employee = e.id_employee
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
LEFT JOIN public.physical_resources pr ON al.id_physical_resource =
pr.id_physical_resource

```

```

LEFT JOIN public.information_resources ir ON
al.id_information_resource = ir.id_information_resource
WHERE es.status_type != 'Активная работа';

```

Представление 8. «Статистика доступов по статусам»

Список атрибутов, отображаемых в рамках представления «Статистика доступов по статусам»:

- Тип статуса (отношение «employees_statuses»);
- Общее число попыток (агрегация по «access_log»);
- Успешных попыток (агрегация по «access_log»);
- Неуспешных попыток (агрегация по «access_log»).

```

CREATE VIEW public.access_statistics_by_status
AS
SELECT
    es.status_type,
    COUNT(*) AS total_attempts,
    COUNT(*) FILTER (WHERE al.result = 'успешно') AS successful,
    COUNT(*) FILTER (WHERE al.result = 'отказано') AS failed
FROM public.access_log al
JOIN public.employees e ON al.id_employee = e.id_employee
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
GROUP BY es.status_type;

```

Представление 9. «Потенциальные прогулы»

Список атрибутов, отображаемых в рамках представления «Потенциальные прогулы»:

- ФИО (отношение «employees»);
- Логин (отношение «employees»);
- Название команды (отношение «teams»);
- Тип статуса (отношение «employees_statuses»);
- Активность (отношение «employees_statuses»);
- Последняя запись в Журнале доступа (дата, ресурс).

```

CREATE VIEW public.potential_absenteeism
AS
SELECT

```

```

    e.full_name,
    e.login,
    t.team_name,
    es.status_type,
    es.employee_active,
    al.event_date AS last_access,
    COALESCE(pr.resource_name, ir.resource_name) AS last_resource
FROM public.employees e
JOIN public.employees_statuses es ON e.id_employee = es.id_employee
LEFT JOIN public.teams t ON e.id_team = t.id_team
LEFT JOIN LATERAL (
    SELECT *
    FROM public.access_log al2
    WHERE al2.id_employee = e.id_employee
    ORDER BY al2.event_date DESC
    LIMIT 1
) al ON TRUE
LEFT JOIN public.physical_resources pr ON al.id_physical_resource =
pr.id_physical_resource
LEFT JOIN public.information_resources ir ON
al.id_information_resource = ir.id_information_resource
WHERE es.status_type = 'Активная работа' AND es.employee_active =
0;

```

Представление 10. «Доступы после смены статуса»

Список атрибутов, отображаемых в рамках представления «Доступы после смены статуса»:

- Логин (отношение «employees»);
- Старый статус — Новый статус (отношение «status_log»);
- Дата смены статуса;
- Первая попытка доступа после смены (дата, ресурс, результат).

```

CREATE VIEW public.access_after_status_change
AS
SELECT
    e.login,
    prev_sl.status_type AS old_status,

```

```

curr_sl.status_type AS new_status,
curr_sl.record_date AS status_change_date,
al.event_date AS first_access_after,
COALESCE(pr.resource_name, ir.resource_name) AS resource_name,
al.result
FROM public.status_log curr_sl
JOIN public.status_log prev_sl
  ON curr_sl.status_code = prev_sl.status_code
  AND prev_sl.record_date < curr_sl.record_date
JOIN public.employees_statuses es ON curr_sl.status_code =
es.status_code
JOIN public.employees e ON es.id_employee = e.id_employee
JOIN public.access_log al
  ON e.id_employee = al.id_employee
  AND al.event_date > curr_sl.record_date
LEFT JOIN public.physical_resources pr ON al.id_physical_resource =
pr.id_physical_resource
LEFT JOIN public.information_resources ir ON
al.id_information_resource = ir.id_information_resource
WHERE curr_sl.status_type IN ('Уволен', 'Отпуск', 'Больничный')
ORDER BY curr_sl.record_date DESC
LIMIT 20;

```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы успешно создана база данных управления доступом в СУБД PostgreSQL. Реализованы все таблицы согласно схеме, установлены связи и индексы, данные заполнены с учетом безопасности (пароли хэшированы). Созданы 10 представлений для удобного анализа прав доступа, нарушений политик, статусов сотрудников и статистики. Работа закрепила практические навыки проектирования и администрирования БД, а также применения базовых механизмов информационной безопасности.